Metropolis–Hastings example: annotated

AUTHOR
Adapted by Patricia Menéndez from
Florian Hartig blogpost

This example is adapted from the original `MHexample` file and is annotated using the notation from the Metropolis–Hastings lecture slides.

1. Simulate test data

We generate data from the linear model

$$Y_i = ax_i + b + \varepsilon_i, \qquad \varepsilon_i \sim N(0, \sigma^2).$$

The unknown parameter vector is

$$\boldsymbol{\theta} = (a, b, \sigma).$$

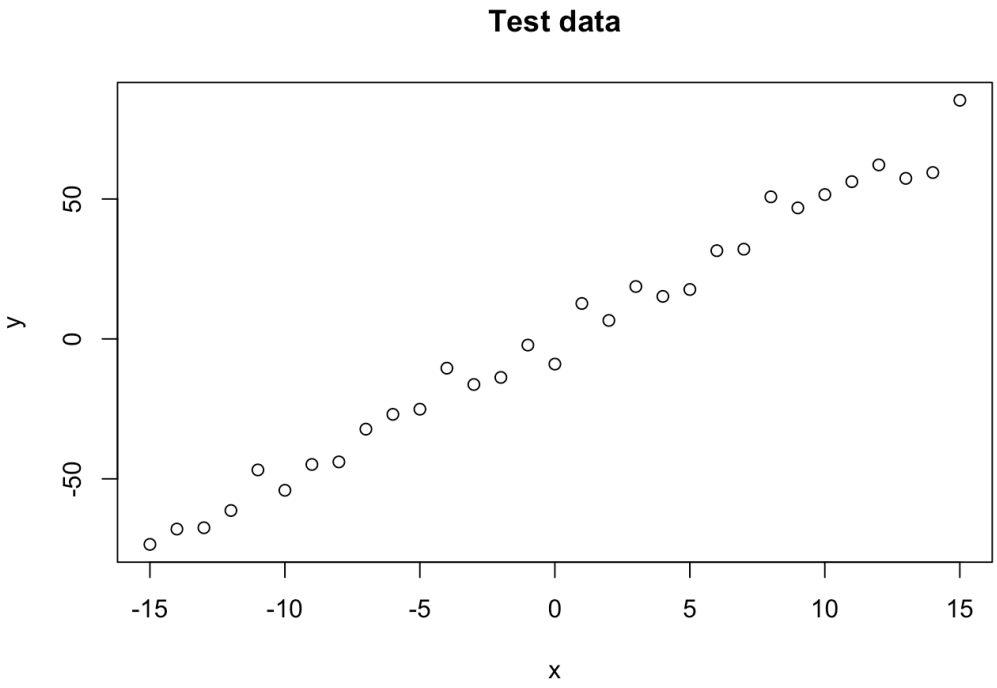
```
trueA <- 5
trueB <- 0
trueSd <- 5
sampleSize <- 31

set.seed(1500)

# x-values (equally space around 0)
x <- (-(sampleSize - 1) / 2):((sampleSize - 1) / 2)

# Data generated from y = ax + b + Normal(0, sigma^2)
y <- trueA * x + trueB + rnorm(
  n = sampleSize,
  mean = 0,
  sd = trueSd
)

plot(x, y, main = "Test data")
```



2. What is the target distribution?

Our goal is to generate samples from the posterior distribution

$$\pi(\boldsymbol{\theta}) = p(\boldsymbol{\theta} \mid \mathbf{y}).$$

Using Bayes' rule,

$$p(\boldsymbol{\theta} \mid \mathbf{y}) = \frac{p(\mathbf{y} \mid \boldsymbol{\theta})p(\boldsymbol{\theta})}{p(\mathbf{y})} \propto p(\mathbf{y} \mid \boldsymbol{\theta})p(\boldsymbol{\theta}).$$

> ⓘ **Key idea**
>
> We **do not need to know the normalising constant** $p(\mathbf{y})$. Metropolis–Hastings only needs the target distribution up to a constant of proportionality.
>
> So in the algorithm we can evaluate
>
> $$\pi(\boldsymbol{\theta}) = p(\mathbf{y} \mid \boldsymbol{\theta})p(\boldsymbol{\theta}).$$

3. Proposal distribution q

Suppose the current value of the chain is

$$\boldsymbol{\theta}^{(t)}.$$

We generate **one candidate value $\boldsymbol{\theta}'$** using the proposal distribution

$$\boldsymbol{\theta}' \sim q\left(\boldsymbol{\theta}^{(t)}, \cdot\right).$$

Remember!

$$q(\text{current}, \text{proposed}).$$

Hence

$$q\left(\boldsymbol{\theta}^{(t)}, \boldsymbol{\theta}'\right)$$

is the proposal density for moving from the current value $\boldsymbol{\theta}^{(t)}$ to the proposed value $\boldsymbol{\theta}'$.

In this example we use a random-walk Normal proposal:

$$\begin{pmatrix} a' \\ b' \\ \sigma' \end{pmatrix} \sim N_3 \left(\begin{pmatrix} a^{(t)} \\ b^{(t)} \\ \sigma^{(t)} \end{pmatrix}, \begin{pmatrix} 0.1^2 & 0 & 0 \\ 0 & 0.5^2 & 0 \\ 0 & 0 & 0.3^2 \end{pmatrix} \right).$$

```
proposalfunction <- function(param) {
  rnorm(3, mean = param, sd = c(0.1, 0.5, 0.3))
}
```

> **Why does the proposal ratio disappear here?**
>
> This is a **symmetric random-walk proposal**. Therefore,
>
> $$q(\boldsymbol{\theta}^{(t)}, \boldsymbol{\theta}') = q(\boldsymbol{\theta}', \boldsymbol{\theta}^{(t)}).$$
>
> So the proposal-density terms cancel in the Metropolis–Hastings acceptance probability.

4. Acceptance probability A

In general, the Metropolis–Hastings acceptance probability is

$$A = \min\left\{1, \frac{\pi(\boldsymbol{\theta}')q(\boldsymbol{\theta}', \boldsymbol{\theta}^{(t)})}{\pi(\boldsymbol{\theta}^{(t)})q(\boldsymbol{\theta}^{(t)}, \boldsymbol{\theta}')}\right\}.$$

We can separate this into two pieces:

$$\underbrace{\frac{\pi(\boldsymbol{\theta}')}{\pi(\boldsymbol{\theta}^{(t)})}}_{\text{compare proposed and current values under the target}} \times \underbrace{\frac{q(\boldsymbol{\theta}', \boldsymbol{\theta}^{(t)})}{q(\boldsymbol{\theta}^{(t)}, \boldsymbol{\theta}')}}_{\text{correct if one proposal direction is easier than the reverse}}.$$

Because our proposal is symmetric,

$$\frac{q(\boldsymbol{\theta}', \boldsymbol{\theta}^{(t)})}{q(\boldsymbol{\theta}^{(t)}, \boldsymbol{\theta}')} = 1,$$

and therefore

$$\boxed{A = \min\left\{1, \frac{\pi(\boldsymbol{\theta}')}{\pi(\boldsymbol{\theta}^{(t)})}\right\}.}$$

5. How do we actually make the accept/reject decision?

After calculating A, generate

$$U \sim U(0,1).$$

Then

$$\boxed{\boldsymbol{\theta}^{(t+1)} = \begin{cases} \boldsymbol{\theta}', & U \le A, \\ \boldsymbol{\theta}^{(t)}, & U > A. \end{cases}}$$

So $\boldsymbol{\theta}'$ is only a **candidate**. It becomes the next value of the chain only if it is accepted.

First iteration

We start with

$$\boldsymbol{\theta}^{(0)}.$$

Generate a candidate

$$\boldsymbol{\theta}' \sim q(\boldsymbol{\theta}^{(0)}, \cdot).$$

Calculate

$$A = \min\left\{1, \frac{\pi(\boldsymbol{\theta}')}{\pi(\boldsymbol{\theta}^{(0)})}\right\},$$

and then select $\boldsymbol{\theta}^{(1)}$:

$$\boldsymbol{\theta}^{(1)} = \begin{cases} \boldsymbol{\theta}', & \text{with probability } A, \\ \boldsymbol{\theta}^{(0)}, & \text{with probability } 1 - A. \end{cases}$$

Remember

$$\boxed{\boldsymbol{\theta}^{(0)}} \quad \xrightarrow[\text{using } q]{\text{propose}} \quad \boxed{\boldsymbol{\theta}'} \quad \xrightarrow[\text{probability } A]{\text{accept/reject}} \quad \boxed{\boldsymbol{\theta}^{(1)}}.$$

6. Why can we calculate the posterior ratio if we do not know the posterior exactly?

We know that

$$\pi(\boldsymbol{\theta}) = p(\boldsymbol{\theta} \mid \mathbf{y}) \propto p(\mathbf{y} \mid \boldsymbol{\theta}) p(\boldsymbol{\theta}).$$

Therefore

$$\frac{\pi(\boldsymbol{\theta}')}{\pi(\boldsymbol{\theta}^{(t)})} = \frac{p(\mathbf{y} \mid \boldsymbol{\theta}') p(\boldsymbol{\theta}')}{p(\mathbf{y} \mid \boldsymbol{\theta}^{(t)}) p(\boldsymbol{\theta}^{(t)})},$$

because the unknown normalising constant cancels.

> ⚠ This is why Metropolis–Hastings is useful
>
> We may not be able to **sample directly** from the posterior, and we may not know its normalising constant, but we can often **evaluate something proportional to the posterior** at any proposed parameter value.

7. Log-posterior used in the R code

In the code it is numerically more convenient to work with the **log posterior**:

$$\log \pi(\boldsymbol{\theta}) = \log p(\mathbf{y} \mid \boldsymbol{\theta}) + \log p(\boldsymbol{\theta}).$$

```
# Evaluate the log posterior, up to an additive constant
posterior <- function(param) {
  likelihood(param) + prior(param)
}
```

The priors used in the example are

$$a \sim U(0, 10), \qquad b \sim N(0, 5^2), \qquad \sigma \sim U(0, 10).$$

```
# Evaluate log prior
prior <- function(param) {
  a <- param[1]
  b <- param[2]
  sd <- param[3]

  aprior <- dunif(a, min = 0, max = 10, log = TRUE)
  bprior <- dnorm(b, mean = 0, sd = 5, log = TRUE)
  sdprior <- dunif(sd, min = 0, max = 10, log = TRUE)

  aprior + bprior + sdprior
}
```

For the likelihood,

$$Y_i \mid a, b, \sigma \sim N(ax_i + b, \sigma^2),$$

so

$$p(\mathbf{y} \mid a, b, \sigma) = \prod_{i=1}^{n} p(y_i \mid a, b, \sigma).$$

Taking logs turns the product into a sum:

$$\log p(\mathbf{y} \mid a, b, \sigma) = \sum_{i=1}^{n} \log p(y_i \mid a, b, \sigma).$$

```
# Evaluate log likelihood
likelihood <- function(param) {
  a <- param[1]
  b <- param[2]
  sd <- param[3]
```

```
  pred <- a * x + b
  singlelikelihoods <- dnorm(y, mean = pred, sd = sd, log = TRUE)
  sum(singlelikelihoods)
}
```

8. Connecting the acceptance formula to the code

For the symmetric proposal,

$$A = \min\left\{1, \frac{\pi(\boldsymbol{\theta}')}{\pi(\boldsymbol{\theta}^{(t)})}\right\}.$$

Because `posterior()` returns the **log** of π, we have

$$\frac{\pi(\boldsymbol{\theta}')}{\pi(\boldsymbol{\theta}^{(t)})} = \exp\left[\log \pi(\boldsymbol{\theta}') - \log \pi(\boldsymbol{\theta}^{(t)})\right].$$

Therefore the code uses

```
A <- min(
  1,
  exp(posterior(proposal) - posterior(chain[i, ]))
)
```

Then `runif(1)` generates the $U \sim U(0,1)$ value used to make the random accept/reject decision.

9. Implement the Metropolis–Hastings algorithm

```
run_metropolis_MCMC <- function(startvalue, iterations) {

  # Store theta^(0), theta^(1), ..., theta^(iterations)
  chain <- array(dim = c(iterations + 1, 3))
  chain[1, ] <- startvalue

  for (i in 1:iterations) {

    # Current value: theta^(t)
    current <- chain[i, ]

    # Step 1: propose theta' using q(theta^(t), .)
    proposal <- proposalfunction(current)

    # Step 2: calculate the acceptance probability A
    # q cancels because the random-walk proposal is symmetric
    A <- min(
      1,
      exp(posterior(proposal) - posterior(current))
    )

    # Step 3: U ~ Uniform(0,1)
    U <- runif(1)
```

```
    # Step 4: select theta^(t+1)
    if (U <= A) {
      chain[i + 1, ] <- proposal
    } else {
      chain[i + 1, ] <- current
    }
  }

  chain
}
```

> The whole algorithm in one line

$$\boxed{\text{current} \longrightarrow \text{propose using } q \longrightarrow \text{accept with probability } A \longrightarrow \text{next state}}$$

10. Run the chain

```
set.seed(1)

# Initial value theta^(0) = (a, b, sigma)
startvalue <- c(4, 0, 4)

# Generate 10,000 MCMC iterations
chain <- run_metropolis_MCMC(startvalue, 10000)

# Remove the first 5,000 iterations as burn-in
burnIn <- 5000

# Proportion of iterations at which the chain moved
acceptance <- 1 - mean(duplicated(chain[-(1:burnIn), ]))
acceptance
```

```
[1] 0.6414717
```

11. Posterior samples

After burn-in, the values in the chain are used as approximate samples from the posterior distribution of (a, b, σ).

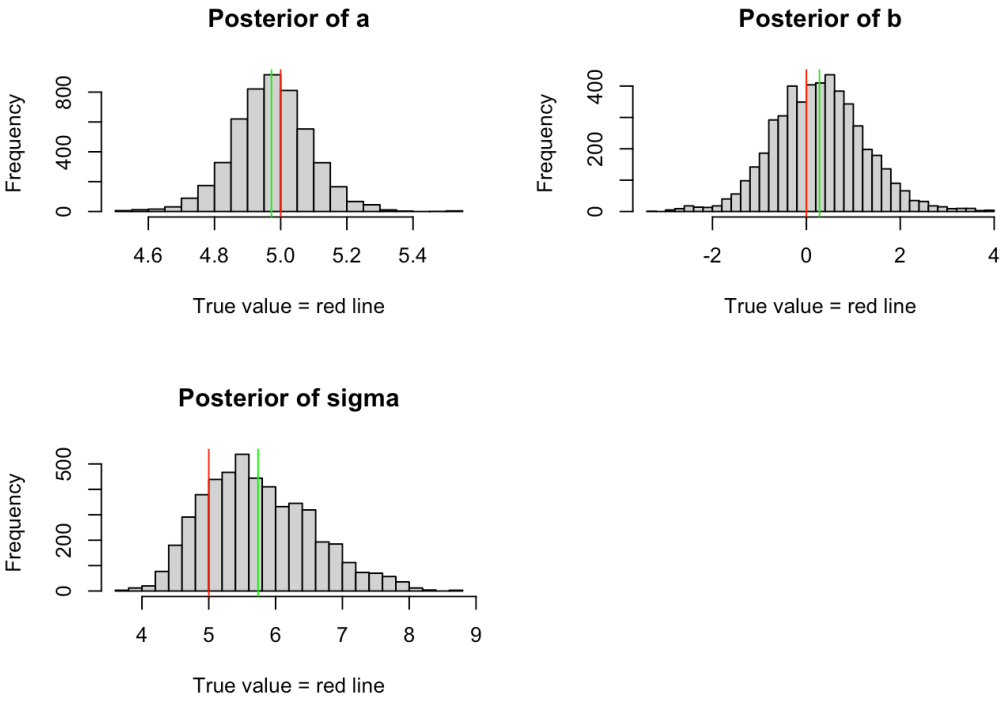
```
par(mfrow = c(2, 2))

hist(
  chain[-(1:burnIn), 1],
  breaks = 30,
  main = "Posterior of a",
  xlab = "True value = red line"
)
abline(v = mean(chain[-(1:burnIn), 1]), col = "green")
abline(v = trueA, col = "red")

hist(
```

```
  chain[-(1:burnIn), 2],
  breaks = 30,
  main = "Posterior of b",
  xlab = "True value = red line"
)
abline(v = mean(chain[-(1:burnIn), 2]), col = "green")
abline(v = trueB, col = "red")

hist(
  chain[-(1:burnIn), 3],
  breaks = 30,
  main = "Posterior of sigma",
  xlab = "True value = red line"
)
abline(v = mean(chain[-(1:burnIn), 3]), col = "green")
abline(v = trueSd, col = "red")
```



The histograms show the **MCMC-generated posterior samples** for the three unknown parameters. The red vertical lines show the true values used to simulate the data, and the green lines show the posterior sample means.

12. Comparison with ordinary linear regression

```
summary(lm(y ~ x))
```

```
Call:
lm(formula = y ~ x)

Residuals:
     Min       1Q   Median       3Q      Max
-10.3580  -3.8445   0.8254   2.4071  10.7585

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
```

```
(Intercept)   0.2710     0.9989   0.271    0.788
x             4.9678     0.1117  44.482   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 5.562 on 29 degrees of freedom
Multiple R-squared:  0.9856,    Adjusted R-squared:  0.9851
F-statistic:  1979 on 1 and 29 DF,  p-value: < 2.2e-16
```

The purpose of this comparison is to check that the MCMC posterior estimates for a and b are broadly consistent with what we would expect from the corresponding linear regression model.